AutoMA: Towards Automatic Model Augmentation for Transferable Adversarial Attacks

Haojie Yuan, Qi Chu, Feng Zhu, Rui Zhao, Bin Liu, and Nenghai Yu

***Abstract*—Recent adversarial attack works attempt to improve the transferability by applying various differentiable transformations on input images. Considering the differentiable transformations and the original model together as a new model, these methods can be regarded as model augmentation that effectively derives an ensemble of models from the single original model. Despite their impressive performance, the model augmentation policies used in these methods are manually designed by experimental attempts, leaving the design of model augmentation policy an open question. In this paper, we propose an Automatic Model Augmentation (AutoMA) approach to find a strong model augmentation policy for transferable adversarial attacks. Specifically, we design a discrete search space that contains various diffierentiable transformations with different parameters and adopt reinforcement learning to search for the strong augmentation policy. The sampled augmentation policies together with the rewards they obtain during the searching process reveal several valuable observations for designing more powerful attacks using model augmentation policy: *1) Augmentation transformations on color space are less effective; 2) The transformation type diversity matters; and 3) Using small distortion for geometric transformations while larger distortion for intensity transformations.* Extensive experiments show that the augmentation policy found by AutoMA achieves superior performance than existing manually designed policies in a wide range of cases.**

***Index Terms*—Adversarial attack, AutoML, transferability.**

I. Introduction

DEEP Neural Networks (DNNs) have achieved great success in various computer vision tasks. such as image classification [1]–[3], object detection [4]–[6], semantic segmentation [7], [8] and so on. However, it has been shown that DNNs are vulnerable to adversarial examples which apply human-imperceptible perturbations on benign inputs, such attacks could succeed on image classification [9], recommendation systems [10], cross-modal learning [11], deep hashing methods [12] and even CAPTCHAs [13]. Studying adversarial attacks helps reveal more nature of DNNs and understand it better. Meanwhile, constructing more powerful adversarial examples helps us better assess the risk of critical applications using DNNs and build stronger baseline for evaluating network robustness and defense methods [14].

Adversarial attacks could be roughly divided into two categories: white-box attacks and black-box attacks. In the white-box attack setting, the attacker has full access to every detail of the target network when generating adversarial examples. On the contrary, attacker in the black-box attack setting could not craft adversarial example with full access to the target model, which is more practical and can better reflect security risk of DNN applications.

Based on the transferability of adversarial examples [15], plenty of works tackle the black-box attack by crafting adversarial example on a proxy white-box model and then transferring it to the target black-box model. Among these methods, attacking an ensemble of models has been shown an effective way to improve transferability [16]. Considering the differentiable transformations and the original model together as a new model, some recent works [17]–[19] have attempted to derive an ensemble of models from a single source model in the way of model augmentation by performing various differentiable transformations on input images and demonstrated its effectiveness for improving transferability of adversarial examples. However, the model augmentation policies used in existing methods are manually designed by experimental attempts, leaving the design of model augmentation policy an open question.

In this paper, we propose an Automatic Model Augmentation (**AutoMA**) approach to find a better model augmentation policy for transferable adversarial attacks. Specifically, we formulate the problem of model augmentation policy searching as a bi-level optimization problem. To solve it efficiently, we first design a model augmentation policy search space, which restricts massive model augmentation policies into a finite discrete space. Each specific augmentation policy in the search space is a combination of multiple augmentation transformation operations. Each augmentation transformation operation has its own operation type and the magnitude range. In order to find an augmentation policy that could generalize on different models, we collect a pool of diverse models. During the search process, we randomly choose one model as substitute white-box model and the remaining models are used as the target black-box model evaluation set at each iteration. The average success attack rate on the evaluation set is regarded as the reward of

Manuscript received 13 June 2021; revised 20 September 2021 and 25 October 2021; accepted 26 October 2021. Date of publication 2 November 2021; date of current version 13 January 2023. This work was supported in part by the National Natural Science Foundation of China under Grants 62002336 and U20B2047, and in part by the SenseTime Research Fund for Young Scholars. The Associate Editor coordinating the review of this manuscript and approving it for publication was Prof. Marco Grangetto. *(Corresponding author: Qi Chu.)*

Haojie Yuan, Qi Chu, Bin Liu, and Nenghai Yu are with the School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230026, China (e-mail: doubihj@mail.ustc.edu.cn; qchu@ustc.edu.cn; flowice@ustc.edu.cn; ynh@ustc.edu.cn).

Feng Zhu and Rui Zhao are with the SenseTime Ltd., Shenzhen 265000, China (e-mail: zhufengx@mail.ustc.edu.cn; fantasticzr2011@gmail.com).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TMM.2021.3124083.

Digital Object Identifier 10.1109/TMM.2021.3124083

1520-9210 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

corresponding augmentation policy. With the designed discrete search space and reward function, a strong augmentation policy could be found by discrete searching algorithm.

The sampled augmentation policies together with corresponding rewards during the searching process provide a rich basis for our study. Through extensive analysis, we have gained the following insights for the transferable adversarial attacks using model augmentation policy: 1) Augmentation transformations on color space are less effective; 2) The transformation type diversity matters; 3) Using small distortion for geometric transformations while larger distortion for intensity transformations.

We also conduct extensive experiments and the experimental results show that adversarial attacks using the model augmentation policy found by AutoMA can achieve superior attacking performance compared to previous manually designed model augmentation policies in a wide range of settings, including attacking normally trained models, adversarially trained models and other defense methods. What's more, the model augmentation policy searched on ImageNet also leads to strong adversarial attack performance on other datasets such as Oxford Flowers [20] and Caltech 101 [21], which further demonstrates the generalization ability of the model augmentation policy searched by our method.

II. Related Work

Transferable Adversarial Attacks: One of the important properties of adversarial examples is *transferability*, i.e. adversarial example crafted for one model could fool a different model with high probability [15], which is frequently utilized for performing black-box adversarial attacks. There exist several sorts of works for improving adversarial transferability. Some works focus on non-targeted attack by lerveraging intermediate feature representations [22], [23] and various regularization terms [24], [25], while others focus on targeted attack by metric learning [26] or attentive 'push' and 'pulls' [27]. Considering the adversarial examples generation as an optimization process, some methods adopt the idea of momentum [16] and Nesterov accelerated gradient [17] into the iterative attacks. Some works focus on ensemble-based mechanism, since attacking an ensemble of multiple substitute models has been shown an effective way to improve transferability [15]. Among them, Ghost networks [28] apply feature-level perturbations to an existing model to potentially create a huge set of diverse models. Another series of works focuses on applying various differentiable transformations on input images to improve transferability of adversarial examples. For example, the diverse inputs method (DI) [18] applies random resizing and padding to the input images at each attack iteration. The scale-invariant method (SI) [17] optimizes the adversarial perturbations over the scale copies of the input images. The diversity-ensemble (DE) method [19] modifies DI and extend it to multi-scale version similar to SI. Aiming to improve the transferability for defense models, the translation-invariant method (TI) [29] attempts to optimize the perturbation over an ensemble of translated images and implements it by convolving the gradient of the original image with a pre-defined gaussian kernel under the assumption that the model are translation-invariant. Considering the differentiable transformations and the original model together as a new model, these methods can be regarded as model augmentation that effectively derives an ensemble of models from the single original model. Our method follows the idea of model augmentation [17], [19] via applying differentiable transformations on input images. Different from existing methods, we aim at automatically searching for a model augmentation policy with good transferability rather than manually designing it.

Adversarial Defenses: Various methods have been proposed to defend against adversarial examples. The most widely applied one is adversarial training [30], [31], which injects adversarial examples into the training data to improve network robustness. Despite its high computational cost, adversarial training, currently, is still one of the most effective defense methods. Some recent works [32], [33] attempt to design more efficient adversarial training methods. Meanwhile, attempts to analyze and regularize Lipschitz norm of DNN could also be applied for adversarial defense [34]. Another line of works focuses on modifying the inputs before feeding them to the DNN model to alleviate the effects of adversarial perturbations, including high-level representation guided denoising [35], random resizing and padding [36] and image compression [37]. Adversarial example detection methods are also studied in recent researches, such as leveraging statistical properties [38], analyzing supportive training examples [39] and designing context consistency rules [40].

Learning Augmentation Policies: Recently, several works have attempted to learn data augmentation policies automatically for improving the performance of DNN models. AutoAugment [41] searches through a pre-defined augmentation space to find the best data augmentation policy. Adversarial AutoAugment [42] extends the fixed augmentation policy to a dynamic schedule during training process. Different from these methods, we aim at learning model augmentation policy for strong transferability of adversarial attacks among different models rather than improving the performance of DNN models. A random generation-evaluation manner is specially designed during our searching process, which is not observed in any other RL-based policy learning methods. The ablation study shows the effectiveness of such the design. Besides, by analyzing a huge amount of sampled augmentation policies for adversarial attacks, we have gained several insights for the transferable adversarial attacks using model augmentation policy, which could serve as basis for future adversarial attack study.

III. Automatic Model Augmentation

A. Notation

White-box Adversarial Attack: Let $\boldsymbol{x}$ and y^t be a benign image and its ground truth label, respectively. Let $J_f(\boldsymbol{x}, y^t)$ be the loss function of the classifier f. The goal of the white-box non-targeted adversarial attack is to find an adversarial example $\boldsymbol{x}^{adv}$ to maximize the loss $J_f(\boldsymbol{x}^{adv}, y^t)$ in the L_p norm bounded perturbations, which can be formalized as:

$$\arg\max_{\boldsymbol{x}^{adv}} J_f(\boldsymbol{x}^{adv}, y^t), \quad \text{s.t. } \left\|\boldsymbol{x}^{adv} - \boldsymbol{x}\right\|_p \leq \epsilon \tag{1}$$

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

where ϵ is the magnitude constraint of adversarial perturbations.

Ensemble-based Attack: Given a set of models $\mathcal{F}$, attacking the ensemble of them by averaging their losses could then be formalized as:

$$\begin{aligned} &\arg\max_{\boldsymbol{x}^{adv}} \tfrac{1}{|\mathcal{F}|} \sum_{f\in\mathcal{F}} J_f(\boldsymbol{x}^{adv}, y^t), \\ &\text{s.t. } \left\|\boldsymbol{x}^{adv} - \boldsymbol{x}\right\|_p \le \epsilon \end{aligned} \tag{2}$$

where $|\mathcal{F}|$ denotes the number of models in $\mathcal{F}$.

Model Augmentation: Applying a set of differentiable transformations $\pi = \{T_1, T_2, \ldots, T_m\}$ on the input image $\boldsymbol{x}$ can derive a set of transformed images $\{T_1(\boldsymbol{x}), T_2(\boldsymbol{x}), \ldots, T_m(\boldsymbol{x})\}$ which can be used to attack the substitute model f_0 to obtain diverse gradients with regard to $\boldsymbol{x}$ for generating adversarial example $\boldsymbol{x}^{adv}$ with better transferability. Considering T_i and the original substitute model f_0 together as a new model $f_i = f_0 \circ T_i$, the transformation set π can be regarded as a model augmentation policy. Then the model set $\mathcal{F}_\pi$ derived from the original substitute white-box model f_0 by applying a model augmentation policy π can be defined as:

$$\begin{aligned} &\mathcal{F}_\pi = \{f_0 \circ T_1, f_0 \circ T_2, \ldots, f_0 \circ T_m\} \\ &f_0 \circ T_i(\boldsymbol{x}) \triangleq f_0(T_i(\boldsymbol{x})) \end{aligned} \tag{3}$$

Automatic Model Augmentation: The goal of automatic model augmentation is to find a model augmentation policy π with best transferability on black-box models, which is formulated as a bi-level optimization problem:

$$\begin{aligned} &\arg\max_{\pi} \tfrac{1}{|\mathcal{G}|} \sum_{g\in\mathcal{G}} J_g(\boldsymbol{x}^*_\pi, y^t), \quad \pi : f_0 \mapsto \mathcal{F}_\pi, \\ &\text{s.t.} \quad \boldsymbol{x}^*_\pi = \arg\max_{\boldsymbol{x}^{adv}} \tfrac{1}{|\mathcal{F}_\pi|} \sum_{f\in\mathcal{F}_\pi} J_f(\boldsymbol{x}^{adv}, y^t) \\ &\qquad\text{s.t.} \quad \left\|\boldsymbol{x}^{adv} - \boldsymbol{x}\right\|_p \le \epsilon \end{aligned} \tag{4}$$

where $\boldsymbol{x}^*_\pi$ is the adversarial example generated by attacking an ensemble of models $\mathcal{F}_\pi$. $\mathcal{G}$ denotes a set of target black-box models. Solving such optimization problem directly is computational unrealistic. We adopt searching algorithms to derive sub-optimal solutions. In following sections, we firstly design a discrete search space for augmentation policy π in Sec. III-B and then describe the detailed search process in Sec. III-C.

B. Search Space

Inspired by existing manually designed augmentation policies [17], [19], augmentation policy π is designed to contain m separate differentiable augmentation transformations in our policy search space. Thus, the input image will be independently transformed to m different versions. Each augmentation transformation is characterized by 2 hyper-parameters: (1) the type of transformation, (2) the magnitude range of the transformation.

Specifically, the augmentation transformation types in our search space contain geometric transformations such as Resize & Padding, Translation, Rotation, Horizontal Flip, etc., intensity transformations such as scaling, adding Gaussian noise, sample pairing, etc., and color space augmentation transformations including Adjusting Hue, Adjusting Saturation, Adjust Brightness, etc. The magnitude range is used for sampling a specific magnitude for the corresponding transformation. We provide detail descriptions of each augmentation method and corresponding magnitude range presented in our search space in Table I. To find suitable ranges for different augmentation transformations, we split the maximum magnitude range of each augmentation transformation into 10 slices uniformly. Augmentation methods are categorized as Geometric Transformations, Intensity Transformations and Color Space Transformations.

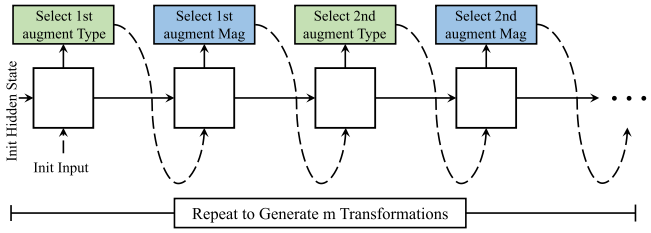


Fig. 1. Illustration of how our controller samples a policy.

C. Search Process

Given the search space and optimization objective, different searching techniques could be used for optimization. Following [41], we adopt Reinforcement Learning (RL) to search for strong augmentation policy from the designed search space. At each iteration, we first use the controller to sample several policies from the search space and calculate their corresponding rewards. Then the controller is updated according to rewards using the policy gradient algorithm.

Policy Sampling: Following [41], [42], we use a one-layer LSTM [44] with 1000 hidden units as the controller RNN. As shown in Fig. 1, the controller RNN needs to repeat $2\,m$ times to sample an augmentation policy with m transformations. The outputs of controller RNN are decoded as the transformation type and the magnitude range by their individual decoder alternately.

Reward Calculating: The reward function is essential to the performance of the searched results. In our implementation, the reward $r(\pi)$ for the given augmentation policy π is defined as the average attack success rate on black-box models with the adversarial examples generated using π:

$$r(\pi) = \frac{1}{|\mathcal{G}|} \sum_{g\in\mathcal{G}} r_g(\mathcal{X}^*_\pi, \mathcal{Y}^t) \tag{5}$$

where $\mathcal{X}^*_\pi = \{\boldsymbol{x}^*_{\pi,i}\}_{i=1}^N$ is the set of adversarial examples crafted on the dataset $\mathcal{X} = \{\boldsymbol{x}_i\}_{i=1}^N$ using augmentation policy π and $\mathcal{Y}^t = \{y_i^t\}_{i=1}^N$ denotes the corresponding ground truth labels. $r_g(\mathcal{X}^*_\pi, \mathcal{Y}^t)$ is the attack success rate of $\mathcal{X}^*_\pi$ on model g:

$$r_g(\mathcal{X}^*_\pi, \mathcal{Y}^t) = \frac{\sum_{i=1}^N \mathbf{1}(g(\boldsymbol{x}^*_{\pi,i}) \neq y_i^t)}{N} \tag{6}$$

Detail of the dataset $\mathcal{X}$ used for searching is described in Sec. V-A. Theoretically, $\mathcal{G}$ should contain as many as possible models to better evaluate the transferability of adversarial examples and improve the generalization capability of learned augmentation policy. While considering the difficulty in collecting too many models, we propose to learn model-irrelevant

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MAXIMUM MAGNITUDE RANGE (FOR IMAGE SIZE 299X299) AND DETAILED DESCRIPTION OF OPERATIONS. THE MAXIMUM RANGE IS UNIFORMLY DIVIDED INTO 10 SLICES. '$-$' MEANS THAT THE OPERATION DOESN'T USE MAGNITUDE INFORMATION

Operation Type	**Maximum Range for magnitude *m***	**Description**
Resize&Padding	[299,1]	Resize image to $m \times m$ and randomly pad the resized image to original size.
Translation	[0,299]	Translate the image in random direction by m pixels.
Rotation	[0,90]	Rotate the image to random direction by m degrees.
Horizontal Flip	$-$	Flip the image horizontally.
Gaussian Noise	[0,1]	Add Gaussian noise with mean 0 and variance m on image normalized to [0,1].
Scaling	[0,10]	Scale the image with factor of $2^{\frac{-m}{2}}$.
Sample Pairing	[0,0.5]	Mixup [43] the origin image with another random image, the weight of random image is m and the weight of origin image is $1-m$.
Adjust Hue	[-10,10]	Transform image to HSV space, adjust Hue vector of image to 1.1^m times then transform back to RGB space.
Adjust Saturation	[-10,10]	Transform image to HSV space, adjust Saturation vector of image to 1.1^m times then transform back to RGB space.
Adjust Brightness	[-10,10]	Transform image to HSV space, adjust Value vector of image to 1.1^m times then transform back to RGB space.

augmentation policies based on a small set of models in a random generation-evaluation manner. Specifically, we first build a model pool $\mathcal{M}$ containing several diverse models. Then at each iteration of the search process, we randomly select one model from $\mathcal{M}$ as the proxy white-box model f_0 for adversarial examples generation and take the remaining models in $\mathcal{M}$ as a set of target black-box models $\mathcal{G}$ for reward calculation.

Adversarial example $\boldsymbol{x}_\pi^*$ in (6) could be generated by (2) using any existing adversarial attack methods. In our implementation, we adopt the commonly used momentum iterative gradient-based method (MIM) [16] for searching. we adopt the commonly used momentum iterative gradient-based method (MIM) [16] during the search process as follows:

$$\begin{aligned} \boldsymbol{g}_{t+1} &= \mu \cdot \boldsymbol{g}_t + \frac{\nabla_{\boldsymbol{x}} \frac{1}{m} \sum_{f \in \mathcal{F}_\pi} J_f(\boldsymbol{x}_t^{adv}, y^t)}{\|\nabla_{\boldsymbol{x}} \frac{1}{m} \sum_{f \in \mathcal{F}_\pi} J_f(\boldsymbol{x}_t^{adv}, y^t)\|_1}, \\ \boldsymbol{x}_{t+1}^{adv} &= \boldsymbol{x}_t^{adv} + \alpha \cdot \operatorname{sign}(\boldsymbol{g}_{t+1}), \end{aligned} \tag{7}$$

where $\boldsymbol{g}_t$ stands for the accumulated gradient in the tth iteration, $\boldsymbol{g}_0 = 0$. μ is the momentum factor for gradient accumulation. α is the step size.

Controller Updating: Since the defined reward function is non-differentiable, we utilize Policy Gradient method [45] to update the controller. Following [41], we adopt the Proximal Policy Optimization algorithm (PPO) [46] which is an improved version of Policy Gradient [45]. For hyper-parameters of PPO, we set the learning rate to 0.00001, baseline weight decay rate to 0.9 and the entropy penalty weight to 0.0002. The overall searching process is shown in Algorithm 1.

Algorithm 1: Search Process

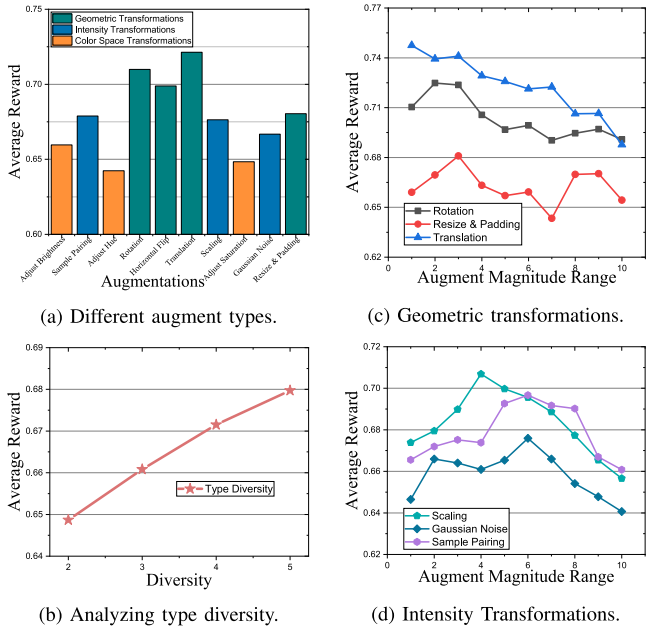
Hyperparameters: Total search step N
Input: Model pool $\mathcal{M}$ for search, searching dataset $\mathcal{X}$
Output: Learned model augmentation policy π^*.
Set step counter $j = 0$, max reward $r_{\max} = 0$;
Initialize controller;
while $j < N$ **do**
  Randomly choose a model from $\mathcal{M}$ as the substitute white-box model f_0, leave the remaining models in $\mathcal{M}$ as the target black-box model set $\mathcal{G}$;
  Sample policy π_j with controller;
  for $\boldsymbol{x}_i$ in $\mathcal{X}$ **do**
    Solve inner maximization in (4) with π_j, $\boldsymbol{x}_i$ and f_0 to generate adversarial example $\boldsymbol{x}_{\pi_j,i}^*$;
  end for
  Collect generated adversarial examples as $\mathcal{X}_{\pi_j}^*$;
  Calculate reward $r(\pi_j)$ in (5) with $\mathcal{X}_{\pi_j}^*$ and $\mathcal{G}$;
  Update controller using PPO with $r(\pi_j)$ and π_j;
  if $r(\pi_j) > r_{\max}$ **then**
    $\pi^* \leftarrow \pi_j$; $r_{\max} \leftarrow r(\pi_j)$;
  end if
end while
Return π^*

IV. ANALYSIS OF AUGMENTATION POLICIES

Since model augmentation policies used in existing methods are manually designed, there are no principles to guide us in designing strong augmentation policies for transferable adversarial attacks. Fortunately, the sampled augmentation policies together with corresponding rewards they obtain during the search process could provide a rich basis for our study. Thus, we conduct extensive analysis on the augmentation policies sampled during the search process and find answers to the several key questions for designing augmentation policies with strong transferability:

1) What kind of augmentation transformation is useful? To answer this question, for each augmentation transformation type, we calculate the average reward of the policies containing it as the effectiveness indicator. The results are shown in Fig. 2(a). It can be seen that the geometric transformations are more effective than intensity augmentation transformations and color space augmentation transformations. Another interesting phenomenon is that adjusting Hue, Saturation, Brightness (corresponding to distortion in HSV space) obtain low average rewards, which indicates that the color space augmentation transformations are ineffective for tranferable adversarial attacks.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.



(a) Different augment types. (c) Geometric transformations.

(b) Analyzing type diversity. (d) Intensity Transformations.

Fig. 2. Analysis of the sampled augmentation policies during the search process.

2) Do we need the type diversity in augmentation policy? Here, the type diversity is simply defined as the number of different transformation types appear in the augmentation policy. We calculate the average reward of the augmentation policies with the same type diversity to analyze the relationship between average reward and the type diversity of the augmentation policy. As shown in Fig. 2(b), the average reward grows along with the increase of the type diversity, which shows that the transformations type diversity in the augmentation policy does matter for improving the transferability.

3) What is the suitable magnitude range for the specific augmentation transformation type? For each augmentation transformation with the specific type and magnitude range level, we calculate the average reward of the augmentation policies containing it as the effectiveness indicator. Fig. 2(c) and Fig. 2(d) show the relationship between the average reward and the magnitude range level of geometric augmentation transformation and intensity augmentation transformations respectively. We can see that as the magnitude range level increases, the the average rewards of geometric transformations have a downward trend, while the average rewards of intensity transformations show an evident rise first and then decrease. Since the magnitude range level is corresponding to the distortion level compared to original images, we can conclude that the small distortion should be kept for geometric transformations while the larger distortion is more suitable for intensity transformations.

The details of the best augmentation policy found by AutoMA are illustrated in Fig. 3, which is also in accord with the above findings.

V. Experiments

A. Experiment Settings

Searching Dataset: The dataset $\mathcal{X}$ we used for policy searching is ImageNet [47] validation set, which contains 50,000 images. For efficiency, we randomly select 1000 images during searching. Note that the dataset we used for searching has no overlap with the dataset used for evaluation below.

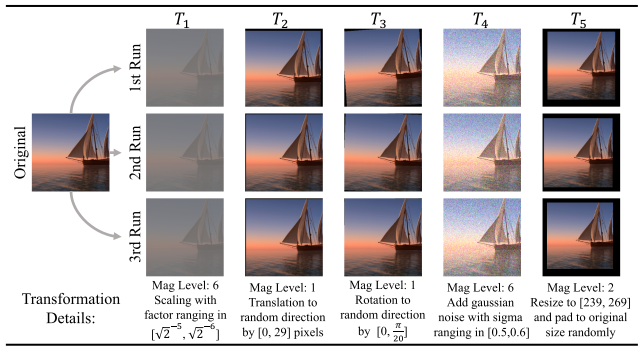


Fig. 3. Details of the augmentation policy found by AutoMA with $m = 5$. Different transformed images could be generated in different attack iterations due to transformation randomness.

Evaluation Dataset: Following [19], [29], we use the commonly used test dataset[1] in NeurIPS-2017 adversarial competition for evaluation. It contains 1,000 ImageNet-compatible images.

Searching Models: The model pool $\mathcal{M}$ we used for searching includes ResNet18 [2], AlexNet [48], SqueezeNet [49], VGG16 [50], ShuffleNet [51], MobileNet [52], ResNeXt50 [53] and MnasNet [54]. To verify that the learned policy does not overfit models used for searching, we adopt an entirely different model set for evaluation as below.

Evaluation Models: Following [16], [18], [19], [29], we adopt 4 normally trained models for adversarial example crafting and evaluation: Inception-v3 (Inc-v3) [55], Inception-v4 (Inc-v4), Inception-ResNet-v2 (IncRes-v2) [56], and ResNet-v2-152 (Res-v2) [57]. We also adopt 3 ensemble adversarially trained models: ens3-adv-Inception-v3 (Inc-v3$_{ens3}$), ens4-adv-Inception-v3 (Inc-v3$_{ens4}$), and ens-adv-InceptionResNet-v2 (IncRes-v2$_{ens}$) [31]. These models are used for attack evaluation only. Moreover, we also include several other defense methods: High-level representation guided denoiser (HGD) [35], Random resizing and padding (R&P) [36], rank-3 submission[2] (NIPS-r3) in the NeurIPS 2017 adversarial competition and two very recent defenses: Feature Distillation (FDistill) [58] and Feature Denoising (FDenoise) [59].

Searching Cost: Searching cost when transformation number $m = 5$ is about 46 GPU hours on NVIDIA TITAN-XP. Larger m will result in longer searching time due to the heavier cost for generating adversarial example. Each increase of m value by 1 would cause about 4 hours additional searching cost.

Evaluation Budgets: We mainly compare the learned augmentation policy with existing manually designed augmentation policies SI [17] and DE [19] in the following experiments. For SI and DE, we follow their original settings with the number of the scale copies as 5. For fair comparison, we set the number of augmentation transformations to $m = 5$ in our augmentation

[1][Online]. Available: https://github.com/tensorflow/cleverhans/tree/master/cleverhans_v3.1.0/examples/nips17_adversarial_competition

[2][Online]. Available: https://github.com/anlthms/nips-2017/tree/master/mmd

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ATTACK SUCCESS RATE (%) OF DIFFERENT MODEL AUGMENTATION POLICIES BASED ON TIM. THE ADVERSARIAL EXAMPLES ARE TESTED ON BOTH NORMALLY TRAINED AND ADVERSARIALLY TRAINED MODELS. * INDICATES WHITE-BOX ATTACKS

Model	Attack	Inc-v3	Inc-v4	IncRes-v2	Res-v2	Inc-v3$_{ens3}$	Inc-v3$_{ens4}$	IncRes-v2$_{ens}$	GPU hours
Inc-v3	TIM	97.6*	45.9	35.2	36.1	35.0	36.0	27.1	**0.05**
	SI-TIM	97.8*	65.0	55.6	52.4	57.5	58.3	44.7	0.13
	DE-TIM	**98.2***	76.4	68.0	64.7	70.1	70.3	58.0	0.14
	ours-TIM	97.5*	**80.7**	**74.3**	**69.3**	**74.8**	**74.3**	**63.6**	0.14
Inc-v4	TIM	56.7	98.5*	40.1	39.5	37.9	39.4	30.6	**0.09**
	SI-TIM	72.4	**98.6***	57.7	54.4	59.7	59.8	50.9	0.23
	DE-TIM	83.5	98.0*	73.3	68.3	71.1	69.2	59.6	0.23
	ours-TIM	**86.8**	98.1*	**78.8**	**71.4**	**76.0**	**75.5**	**67.4**	0.23
IncRes-v2	TIM	65.5	60.7	99.5*	49.1	52.6	52.8	49.0	**0.09**
	SI-TIM	80.2	76.6	**99.6***	70.0	77.7	77.0	73.3	0.24
	DE-TIM	84.9	83.5	98.0*	76.3	79.8	79.5	78.2	0.24
	ours-TIM	**87.2**	**85.6**	98.3*	**79.7**	**84.2**	**82.9**	**82.8**	0.24
Res-v2	TIM	45.6	39.5	33.5	97.1*	41.4	39.2	33.0	**0.10**
	SI-TIM	55.3	52.1	47.7	**97.3***	56.1	53.8	47.2	0.28
	DE-TIM	**75.0**	67.6	67.1	96.5*	71.7	73.6	67.3	0.28
	ours-TIM	**75.0**	**71.2**	**69.3**	97.0*	**73.4**	**74.6**	**68.2**	0.29

TABLE III
ATTACK SUCCESS RATE (%) OF DIFFERENT MODEL AUGMENTATIONS BASED ON MIM. * INDICATES WHITE-BOX ATTACKS

Model	Attack	Inc-v3	Inc-v4	IncRes-v2	Res-v2	Inc-v3$_{ens3}$	Inc-v3$_{ens4}$	IncRes-v2$_{ens}$	GPU hours
Inc-v3	MIM	97.8*	49.4	48.0	39.1	20.0	18.8	10.0	**0.05**
	SI-MIM	97.9*	76.8	74.8	69.0	40.3	40.7	22.8	0.13
	DE-MIM	**98.2***	88.1	86.2	79.0	46.0	46.6	27.6	0.12
	ours-MIM	**98.2***	**91.2**	**91.0**	**82.5**	**49.2**	**49.0**	**29.1**	0.13

TABLE IV
ATTACK SUCCESS RATE (%) OF DIFFERENT MODEL AUGMENTATIONS BASED ON NIM. * INDICATES WHITE-BOX ATTACKS

Model	Attack	Inc-v3	Inc-v4	IncRes-v2	Res-v2	Inc-v3$_{ens3}$	Inc-v3$_{ens4}$	IncRes-v2$_{ens}$	GPU hours
Inc-v3	NIM	98.8*	57.5	54.8	43.8	19.6	18.6	10.4	**0.05**
	SI-NIM	98.8*	75.9	72.9	62.8	35.5	34.9	19.1	0.13
	DE-NIM	**99.0***	85.8	81.3	74.3	37.6	35.5	19.1	0.12
	ours-NIM	98.8*	**88.4**	**86.4**	**80.2**	**41.5**	**39.3**	**21.8**	0.13

policy, which ensures all these policies have the same computation budgets for generating adversarial examples. Meanwhile, we report evaluation cost on whole evaluation dataset in terms of GPU hours in all tables, extra searching cost is reported in previous section. Evaluation computational cost are evaluated on NVIDIA TITAN-XP. The results show that, put aside once-for-all searching cost, our augmentation policy requires roughly the same computation resources as manually designed augmentation policies.

Hyper-Parameters: We use L_∞ norm as magnitude constraint of adversarial perturbation in this work, which is a common practice in adversarial attack studies [16], [17], [19]. Following the settings in [19], we set maximum L_∞ perturbation $\epsilon = 16$, number of iteration $T = 10$, step size $\alpha = 1.6$, and momentum $\mu = 1.0$. For TIM, the Gaussian kernel size is set to 15×15.

B. Attacking a Single Model

Normally Trained Models: Here we report the attack success rate on normally trained models using the learned policy and other existing manually designed augmentation policies SI [17] and DE [19] based on TIM [29]. The models used for generating adversarial examples include Inc-v3, Inc-v4, IncRes-v2, and Res-v2. Results are shown in Table II. We can see that all model augmentation policies perform similarly under white-box setting. Under black-box setting, our learned augmentation policy consistently outperforms other manually designed model augmentation policies.

We also perform evaluation based on MIM [16] and NIM [17] to demonstrate the effectiveness of our policy on different base attacks. The results are shown in Table III and IV respectively. Here, we only show attack results using adversarial examples crafted on Inc-v3 for compactness. We can find that the performance of our learned augmentation policy is consistently superior than other manually designed policies on different base attacks, which further demonstrates the effectiveness of our policy.

Adversarially Trained Models: To further demonstrate the effectiveness of the augmentation policy found by AutoMA, we evaluate it on 3 adversarially trained models: Inc-v3$_{ens3}$, Inc-v3$_{ens4}$, and IncRes-v2$_{ens}$. As shown in Table II–Table IV, the overall attack performance drops on adversarially trained models as expected and our policy still outperforms other manually designed augmentation policies on different base attacks. Note that all the models used during the search process are normally trained. The superior performance on adversarially trained

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

TABLE V
ATTACK SUCCESS RATE (%) OF DIFFERENT MODEL AUGMENTATION POLICIES ON DEFENSE METHODS. ADVERSARIAL EXAMPLES ARE CRAFTED BY ATTACKING ENSEMBLE OF 4 NORMALLY TRAINED MODELS: INC-V3, INC-V4, INCRES-V2, AND RES-V2

Attack	Inc-v3$_{ens3}$	Inc-v3$_{ens4}$	IncRes-v2$_{ens}$	HGD	R&P	NIPS-r3	FDistill	FDenoise	AVG	GPU hours
TIM	76.1	74.8	70.5	74.4	67.5	71.0	73.0	25.9	66.65	**0.35**
SI-NI-TI-DIM	87.2	85.6	77.7	82.3	81.6	84.5	87.8	29.3	76.99	0.92
DE-TIM	91.2	90.7	88.2	90.5	90.1	91.1	92.7	29.7	83.03	0.91
ours-TIM	93.0	93.2	90.7	91.2	90.4	92.0	94.1	33.1	84.71	0.92
RF-DE-TIM	94.7	94.5	89.1	93.2	92.7	93.9	96.9	30.1	85.64	0.92
RF-ours-TIM	**98.5**	**98.4**	**96.7**	**97.6**	**96.5**	**97.6**	**98.7**	**33.3**	**89.66**	0.92

TABLE VI
ATTACK SUCCESS RATE (%) OF DIFFERENT MODEL AUGMENTATIONS ON SEVERAL OTHER DEFENSE METHODS. EVALUATION COMPUTATIONAL COST ARE SAME AS TABLE II

Model	Attack	HGD	R&P	NIPS-r3	FDistill	FDenoise
Inc-v3	TIM	25.3	25.0	27.9	59.1	24.0
	SI-TIM	36.3	34.6	40.0	72.1	25.6
	DE-TIM	61.2	59.3	64.2	81.5	26.4
	ours-TIM	**65.7**	**62.9**	**68.1**	**84.7**	**28.2**
Inc-v4	TIM	29.1	28.2	30.4	61.3	23.6
	SI-TIM	41.9	39.5	42.5	72.7	26.0
	DE-TIM	64.2	63.4	65.1	81.6	25.8
	ours-TIM	**69.6**	**68.0**	**71.0**	**82.1**	**27.5**
IncRes-v2	TIM	44.7	44.9	46.9	67.7	24.6
	SI-TIM	60.6	59.4	62.7	79.7	28.4
	DE-TIM	**80.0**	79.3	80.1	85.4	28.0
	ours-TIM	79.9	**81.6**	**83.7**	**87.2**	**29.1**
Res-v2	TIM	32.5	31.9	34.3	58.8	24.2
	SI-TIM	45.2	43.4	48.4	67.9	26.3
	DE-TIM	66.8	66.3	70.1	81.7	28.1
	ours-TIM	**67.1**	**67.6**	**71.7**	**82.3**	**28.5**

models demonstrates the transferability of the learned augmentation policy.

Other Defense Methods: Besides evaluating on normally trained and adversarially trained models, we also consider several other defense methods. Evaluation results on different defense methods are shown in Table VI. Similar to results on adversarially trained models, our learned augmentation policy outperforms other model augmentation policies, which further demonstrates the effectiveness of the model augmentation policy found by our AutoMA. Note that all the black-box attack performances on Feature Denoise are relatively low, since it is a very strong defense that 2000-iteration white-box PGD attack could only achieve success rate of 57.4% [59].

C. Attacking an Ensemble of Models

Standard Benchmark: More powerful adversarial examples could be crafted with model augmentation policies on ensemble of models. Following [19], we attack the ensemble of Inc-v3, Inc-v4, IncRes-v2 and Res-v2, and evaluate performance on adversarially trained models and several other defense methods. Evaluation results are shown in Table V. It shows Region Fitting (RF) designed for DE [19] could also boost the performance of our policy. Though state-of-the-art method RF-DE-TIM achieves pretty high performance with the average attack success rate of 85.64%, our learned model augmentation policy could still further improve it by 4.02% to reach 89.66%, which is a remarkable improvement since performance against most defense methods is even comparable to the white-box cases.

TABLE VII
COMPARISON WITH DIFFERENT MODEL AUGMENTATION POLICIES. ADVERSARIAL EXAMPLES ARE CRAFTED BY ATTACKING THE ENSEMBLE OF MODELS USED FOR SEARCHING

Attack	Inc-v3	Inc-v4	IncRes-v2	Res-v2	GPU hours
TIM	55.9	56.0	36.2	40.5	**0.23**
SI-TIM	60.9	60.0	40.7	47.8	0.81
DE-TIM	65.6	72.2	49.1	54.9	0.82
ours-TIM	**78.9**	**81.5**	**65.2**	**64.9**	0.82

Attacking an Ensemble of Models Used for Searching: For fair comparison, the models used for searching have no overlap with models used for evaluation in the above experiments. However, one may argue that our policy leverages information from more models, since manually designed augmentation policies do not use the information from the searching models. Thus, for fairness concern on amount of models when comparing with other augmentation policies, we conduct experiments on attacking the ensemble of models used for searching. Under this setting, we make sure that models used for each augmentation policy are exactly the same. The experimental results are shown in Table VII. Since our policy is searched on these models, the performance of our policy significantly outperforms SI and DE as expected. It means that given the same models, our method could better utilize them to generate more powerful adversarial examples compared to other manually designed augmentation polices. Besides, as demonstrated in the previous experiments, the augmentation policy learned from the searching models could also perform well on models that are unseen during searching process. In the Sec. V-E, we will show the effectiveness of the learned policy on other datasets.

D. Comparison With Other Model Augmentation

Different from model augmentation by performing differentiable augmentation transformations on input images, GhostNet [28] applies feature-level perturbations on an existing model to derive diverse models, which is a heuristic way of model augmentation. The number of branch copies S and the number of iterations L are two key hyper-parameters in GhostNet. For fair comparison, we set $S = 5$ and $L = 10$ for GhostNet, which ensures that the computation budget of generating adversarial examples for GhostNet is roughly the same as our method. The comparison results are shown in Table VIII. It can be seen that GhostNet achieves limited performance gain compared to baseline while our method shows much better performance compared to GhostNet.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
COMPARISON WITH DIFFERENT TYPE OF MODEL AUGMENTATION METHODS. ADVERSARIAL EXAMPLES ARE CRAFTED ON INC-V3

Attack	Inc-v3	Inc-v4	IncRes-v2	Res-v2	GPU hours
TIM	97.5*	45.9	35.2	36.1	**0.05**
GhostNet-TIM	90.2*	55.5	44.9	41.1	0.15
ours-TIM	**97.6***	**80.7**	**74.3**	**69.3**	0.13

TABLE IX
DETAILS OF FGVC DATASETS AND MODEL TOP 1 ACCURACY

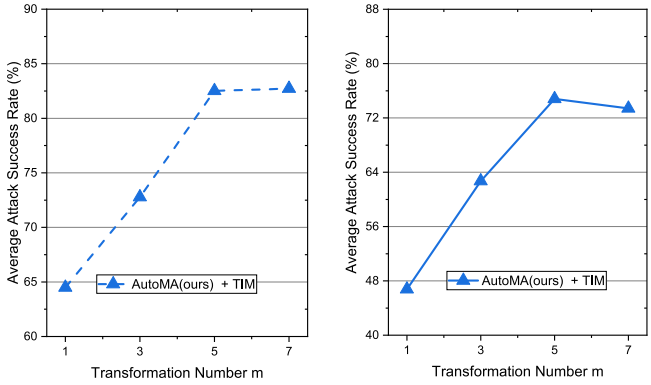
Dataset	Class	Model	Top1-acc
Oxford Flowers	102	Inc-v3	93.97
		Inc-v4	93.51
Caltech 101	101	Inc-v3	92.74
		Inc-v4	91.73

TABLE X
TRANSFERABILITY OF THE POLICY SEARCHED ON IMAGENET. ATTACK SUCCESS RATE (%) ON FGVC DATASETS IS REPORTED BY ATTACKING INC-V3 AND TESTING ON INC-V4

Attack	Oxford 102 Flowers		Caltech101	
	ASR	GPU hours	ASR	GPU hours
TIM	58.3	**0.07**	32.4	**0.07**
SI-TIM	71.6	0.15	56.9	0.15
DE-TIM	85.3	0.15	63.4	0.16
ours-TIM	**86.5**	0.15	**66.9**	0.15

TABLE XI
TARGETED ATTACK SETTINGS. ADVERSARIAL EXAMPLES ARE CRAFTED BY ATTACKING ENSEMBLE OF 4 NORMALLY TRAINED MODELS: INC-V3, INC-V4, INCRES-V2, AND RES-V2

Attack	Inc-v3$_{ens3}$	Inc-v3$_{ens4}$	IncRes-v2$_{ens}$	GPU hours
TIM	0.5	0.7	0.6	**0.33**
SI-TIM	3.4	3.7	2.4	0.91
DE-TIM	5.1	5.0	4.2	0.91
ours-TIM	**5.7**	**5.5**	**4.8**	0.91



(a) On normally trained models. (b) On adv trained models.

Fig. 4. Ablation experiments on numbers of transformation functions. Average attack success rate (%) on both normally trained and adversarially trained models are reported.

E. Transferring to Other Datasets

So far, all evaluation experiments are conducted on ImageNet compatible dataset, which is similar to the dataset we used for policy searching. Though there is no overlap between the evaluation dataset and the searching dataset, whether the learned policy transfers well to other datasets still remains a question. Thus, we also evaluate our augmentation policy learned from ImageNet dataset on Fine Grained Visual Categorization (FGVC) datasets, including Oxford 102 Flowers [20] and Caltech 101 [21]. Since there are no publicly available models pre-trained on these datasets, we have to train models from scratch. Without loss of generality, we train inception-V3 as attack substitute model and inception-V4 as evaluation model. All these models are trained for 100 epochs using adam optimizer with initial learning rate 0.0001. We use Inception pre-processing for training on both datasets. The top 1 accuracy of these models on clean examples is shown in Table IX.

Evaluation results of different augmentation policies on FGVC models are listed in Table X. Compared to other manually designed model augmentation polices, our learned augmentation policy still shows superior performance on datasets that are quite different from the one used for searching. These evaluation results demonstrate that our model augmentation policy can transfer well to other datasets. Such property indicates that augmentation policy found by our algorithm could serve as a strong evaluation method for model robustness evaluation.

F. Targeted Attack Scenarios

All Experiments above are conducted under non-targeted attack settings, i.e. the prediction of adversarial example can be arbitrary class except the original one. We also conduct experiments under the targeted attack setting that the adversarial example is expected to be misidentified as a specific class. Table XI shows the experimental results on attacking adversarially trained models under targeted attack setting. We can see that performances of all attack methods drop a lot, since the transferable targeted attack setting is much more difficult [26]. Despite that our method is not specially designed for the targeted attack setting, our policy still outperforms other manually designed policies.

G. Ablation Study

Transformation Number: In this section, we show the experimental results of the proposed AutoMA with different Transformation function number m. We use the average attack success rate of normally trained models and adversarially trained models in Table II separately as evaluation metrics.

As shown in Fig. 4, when attacking normally trained models, the attack success rate grows with transformation function number m, and saturates at $m = 5$. It demonstrates the effectiveness of model ensemble introduced by model augmentation. However, when attacking adversarially trained models, the performance starts to drop at $m = 7$. It's due to the feature space difference between normally trained models and adversarially trained models. Heavy model augmentation could destroy inherent gradient information of substitute model, which could result in performance drop. However, our policy is searched on normally trained models, thus such performance drop are only observed on adversarially trained models.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

TABLE XII
COMPARISON BETWEEN SEARCHED OUT POLICY AND RANDOM POLICY ON NORMALLY TRAINED MODELS. RANDOM AUGMENTATION POLICY RESULTS ARE AVERAGED ON 5 RANDOMLY SAMPLED POLICIES. ATTACK SUCCESS RATE (%) BASED ON TIM IS REPORTED

Model	Policy	Inc-v3	Inc-v4	IncRes-v2	Res-v2	Inc-v3$_{ens3}$	Inc-v3$_{ens4}$	IncRes-v2$_{ens}$	GPU hours
Inc-v3	no aug	**97.6***	45.9	35.2	36.1	35.0	36.0	27.1	**0.05**
	random aug	96.4*	69.6	64.7	57.6	49.8	49.3	35.4	0.14
	searched aug	97.5*	**80.7**	**74.3**	**69.3**	**74.8**	**74.3**	**63.6**	0.14
Inc-v4	no aug	56.7	**98.5***	40.1	39.5	37.9	39.4	30.6	**0.09**
	random aug	78.0	96.3*	69.5	63.9	56.6	55.2	44.1	0.23
	searched aug	**86.8**	98.1*	**78.8**	**71.4**	**76.0**	**75.5**	**67.4**	0.23
IncRes-v2	no aug	65.5	60.7	**99.5***	49.1	52.6	52.8	49.0	**0.09**
	random aug	79.4	75.8	94.2*	70.7	65.3	61.5	58.3	0.23
	searched aug	**87.2**	**85.6**	98.3*	**79.7**	**84.2**	**82.9**	**82.8**	0.24
Res-v2	no aug	45.6	39.5	33.5	**97.1***	41.4	39.2	33.0	**0.10**
	random aug	72.5	67.0	67.2	94.9*	60.2	57.2	50.3	0.29
	searched aug	**75.0**	**71.2**	**69.3**	97.0*	**73.4**	**74.6**	**68.2**	0.29

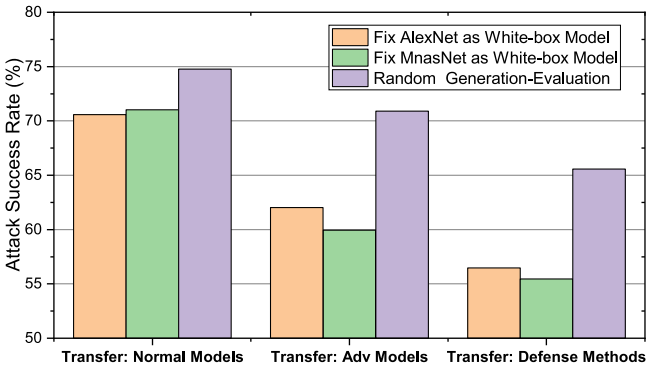


Fig. 5. Ablation experiments on white-box choosing during our searching. We conduct experiments on AlexNet (simple model), MnasNet (complex model) and proposed random generation-evaluation manner. Average attack success rate (%) on white-box model, normally trained and adversarially trained models are reported. Adversarial examples are crafted on inception-v3.

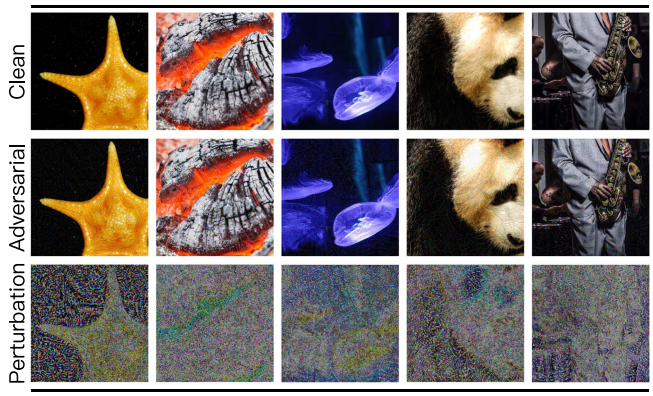


Fig. 6. Visualization of adversarial examples crafted on Inception-v3 using augmentation policy found by AutoMA with the maximum perturbation of each pixel $\epsilon = 16$. Perturbations are scaled 10 times for better visualization.

Another observation is, when we set $m = 1$, the searched out augment transformation is Translation. This result is in accord with the observation that Translation yields the best performance as shown in Fig. 2(a).

Random Augmentation Policy: We then randomize the whole augmentation policy and test its performance. We use $m = 5$ in this section. Results averaged on 5 runs are shown in Table XII. It shows that even random model augmentation policy in our search space could improve attack success rate by a large margin, which highlights the well-design of our search space. Meanwhile, the searched out augmentation could significantly outperform random policies, which demonstrates the effectiveness of our search algorithm. Considering that there exists trade-off between black-box attacks and white-box attacks [16], we also report white-box attack performance. The results show that our method can achieve higher performance under black-box attack setting with only slight performance sacrifice under white-box attack setting.

Random Generation-Evaluation Manner: Finally, we verify our design of random generation-evaluation in searching. We propose to randomly choose proxy model for adversarial example generation during searching to avoid overfitting on certain model. To verify its effectiveness, we conduct the searching process by fixing the model used for adversarial example generation. Here we choose AlexNet to represent simple models and MnasNet to represent complex models. Then we evaluate the searched policy using the same experiment setting in Sec. V-B. The evaluation results of the policy searched by fixing white-box proxy model and the policy searched by random generation-evaluation manner are shown in Fig. 5. We can see the obvious inferior performance when fixing the proxy model, since the found augmentation policy may overfit on specific model. With our random Generation-Evaluation design, the found policy could generalize much better on unseen models. Fig. 6 visualizes some adversarial examples generated by our method.

VI. CONCLUSION

In this paper, we proposed an automatic model augmentation approach to find the strong model augmentation policy for transferable adversarial attacks. By searching over a specially designed search space, a strong augmentation policy could be found. We conduct extensive analysis on the sampled augmentation policies together with the rewards they obtain during the search process and find several insights for the design of augmentation policy with strong transferability. Extensive experiments demonstrate the effectiveness of the augmentation policy found by our AutoMA in various settings. The learned policy has the potential to serve as a strong baseline for evaluating

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

network robustness and defense methods. We make our codes public available at https://github.com/HaojieYuan/autoAdv.

REFERENCES

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Adv. Neural Inf. Process. Syst.*, vol. 25, pp. 1097–1105, 2012.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 48–61.

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, pp. 91–99, 2015.

[5] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 21–37.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.

[7] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2018.

[8] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3431–3440.

[9] C. Szegedy *et al.*, "Intriguing properties of neural networks," *Proc. Int. Conf. Learning Representations*, pp. 252–261, 2014.

[10] Y. Du, M. Fang, J. Yi, C. Xu, J. Cheng, and D. Tao, "Enhancing the robustness of neural collaborative filtering systems under malicious attacks," *IEEE Trans. Multimedia*, vol. 21, pp. 555–565, 2019.

[11] C. Li, S. Gao, C. Deng, D. Xie, and W. Liu, "Cross-modal learning with adversarial samples," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019, pp. 10791–10801.

[12] E. Yang, T. Liu, C. Deng, and D. Tao, "Adversarial examples for hamming space search," *IEEE Trans. Cybern.*, vol. 50, no. 4, pp. 1473–1484, Apr. 2020.

[13] J. Zhang, J. Sang, K. Xu, S. Wu, and J. Yu, "Robust CAPTCHAs towards malicious OCR," *IEEE Trans. Multimedia*, vol. 23, pp. 2575–2587, 2020.

[14] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 39–57.

[15] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1944–1957.

[16] Y. Dong *et al.*, "Boosting adversarial attacks with momentum," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 9185–9193.

[17] J. Lin, C. Song, K. He, L. Wang, and J. E. Hopcroft, "Nesterov accelerated gradient and scale invariance for adversarial attacks," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 4608–4617.

[18] C. Xie *et al.*, "Improving transferability of adversarial examples with input diversity," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2730–2739.

[19] J. Zou, Z. Pan, J. Qiu, X. Liu, T. Rui, and W. Li, "Improving the transferability of adversarial examples with resized-diverse-inputs, diversity-ensemble and region fitting," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 563–579.

[20] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *Proc. Indian Conf. Comput. Vision, Graph. Image Process.*, 2008, pp. 722–729.

[21] L. Fei-Fei, R. Fergus, and P. Perona, "Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2004, pp. 178–178.

[22] Q. Huang, I. Katsman, H. He, Z. Gu, S. Belongie, and S.-N. Lim, "Enhancing adversarial example transferability with an intermediate level attack," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 4733–4742.

[23] N. Inkawhich, W. Wen, H. H. Li, and Y. Chen, "Feature space perturbations yield more transferable adversarial examples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 7066–7074.

[24] W. Zhou *et al.*, "Transferable adversarial perturbations," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 452–467.

[25] W. Wu *et al.*, "Boosting the transferability of adversarial samples via attention," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1161–1170.

[26] M. Li, C. Deng, T. Li, J. Yan, X. Gao, and H. Huang, "Towards transferable targeted attack," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 641–649.

[27] L. Gao, Z. Huang, J. Song, Y. Yang, and H. Shen, "Push & pull: Transferable adversarial examples with attentive attack," *IEEE Trans. Multimedia*, 2021, doi: 10.1109/TMM.2021.3079723.

[28] Y. Li, S. Bai, Y. Zhou, C. Xie, Z. Zhang, and A. L. Yuille, "Learning transferable adversarial examples via ghost networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 7, 2020, pp. 11458–11465.

[29] Y. Dong, T. Pang, H. Su, and J. Zhu, "Evading defenses to transferable adversarial examples by translation-invariant attacks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4312–4321.

[30] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 300–310.

[31] F. Tramér, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble adversarial training: Attacks and defenses," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1908–1921.

[32] A. Shafahi *et al.*, "Adversarial training for free!" in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, pp. 3353–3364, 2019.

[33] H. Zheng, Z. Zhang, J. Gu, H. Lee, and A. Prakash, "Efficient adversarial training with transferable adversarial examples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1181–1190.

[34] S. Amini and S. Ghaemmaghami, "Towards improving robustness of deep neural networks to adversarial perturbations," *IEEE Trans. Multimedia*, vol. 22, pp. 1889–1903, 2020.

[35] F. Liao, M. Liang, Y. Dong, T. Pang, X. Hu, and J. Zhu, "Defense against adversarial attacks using high-level representation guided denoiser," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1778–1787.

[36] C. Xie, J. Wang, Z. Zhang, Z. Ren, and A. Yuille, "Mitigating adversarial effects through randomization," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 2808–2819.

[37] X. Jia, X. Wei, X. Cao, and H. Foroosh, "ComDefend: An efficient image compression model to defend adversarial examples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 6084–6092.

[38] J. Wang *et al.*, "SmsNet: A new deep convolutional neural network model for adversarial example detection," *IEEE Trans. Multimedia*, 2021, doi: 10.1109/TMM.2021.3050057.

[39] G. Cohen, G. Sapiro, and R. Giryes, "Detecting adversarial samples using influence functions and nearest neighbors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 14453–14462.

[40] S. Li *et al.*, "Connecting the dots: Detecting adversarial perturbations using context inconsistency," in *Eur. Conf. Comput. Vis.*, Springer, 2020, pp. 396–413.

[41] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation strategies from data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 113–123.

[42] X. Zhang, Q. Wang, J. Zhang, and Z. Zhong, "Adversarial autoaugment," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 4848–4859.

[43] H. Inoue, "Data augmentation by pairing samples for images classification," 2018, *arXiv:1801.02929*.

[44] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[45] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 13, pp. 1057–1063, 2000.

[46] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[47] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[48] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, pp. 1097–1105, 2012.

[49] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and 0.5 mb model size," 2016, *arXiv:1602.07360*.

[50] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.

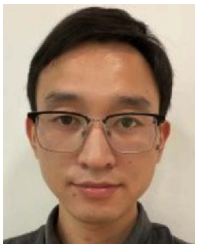
[51] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6848–6856.
[52] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
[53] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1492–1500.
[54] M. Tan *et al.*, "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2820–2828.
[55] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
[56] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, 2017, pp. 4278–4284.
[57] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Eur. Conf. Comput. Vis.*, Springer, 2016, pp. 630–645.
[58] T. Bai, J. Chen, J. Zhao, B. Wen, X. Jiang, and A. Kot, "Feature distillation with guided adversarial contrastive learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 860–868.
[59] C. Xie, Y. Wu, L. v. d. Maaten, A. L. Yuille, and K. He, "Feature denoising for improving adversarial robustness," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 501–509.



Haojie Yuan received the B.S. degree in 2018 from the University of Science and Technology of China, Hebei, China, where he is currently working toward the Ph.D. degree with the School of Cyber Science and Technology. His research interests include media content security and reliable deep learning.



Qi Chu received the B.S. degree in electronic engineering and the Ph.D. degree in information and communication engineering from the University of Science and Technology of China, Hebei, China, in 2014 and 2019, respectively. He is currently an Associate Research Fellow with the School of Cyber Science and Technology, University of Science and Technology of China. His research interests include deep learning, computer vision, and artificial intelligence security.



Feng Zhu received the B.S. degree in electronic engineering and the Ph.D. degree in information and communication engineering from the University of Science and Technology of China, Hebei, China, in 2011 and 2017, respectively. He is currently a Deputy Research Director of Safe City Department, SenseTime. His research interests include computer vision, deep learning, face recognition, and crowd analysis.



Rui Zhao received the bachelor's degree from the University of Science and Technology of China, Hebei, China, in 2010 and the Ph.D. degree from The Chinese University of Hong Kong, Hong Kong, in 2015. He is an Executive Research Director with SenseTime. He is currently a Leading Expert of AI for face recognition and person re-identification. His research interests include deep learning, machine learning, and computer vision.



Bin Liu received the B.S. and M.S. degrees in electrical engineering from the University of Science and Technology of China, Hefei, China, in 1998 and 2001, respectively, and the Ph.D. degree in electrical engineering from Syracuse University, Syracuse, NY, USA, in 2006. He is currently an Associate Professor with the School of Cyber Science and Technology, University of Science and Technology of China. His research interests include computer vision and Internet of Things.



Nenghai Yu received the Ph.D. degree from the University of Science and Technology of China, Hefei, China, in 2004. He is currently a Full Professor with the University of Science and Technology of China. He was a Visiting Scholar with the Institute of Production Technology, Faculty of Engineering, University of Tokyo, Tokyo, Japan, in 1999 and did cooperative research as a Senior Visiting Scholar with the Department of Electrical Engineering, Columbia University, New York, NY, USA, from April to October 2008. His research interests include image processing and video analysis, multimedia communication, media content security, Internet information retrieval, data mining and content filtering, network communication and security.

Authorized licensed use limited to: Nanyang Technological University Library. Downloaded on September 03,2025 at 02:42:50 UTC from IEEE Xplore. Restrictions apply.